

Project Description

SecureVote is a web-based application developed using **HTML, CSS, PHP, and MySQL** with the primary goal of demonstrating the practical application of secure software design principles. The system allows two distinct roles—**Admin** and **Voter**—to interact with elections in a controlled and secure manner.

Admins can create and manage elections, define candidates, and monitor election results, while voters can securely authenticate themselves and cast exactly one vote per election. The system emphasizes **Confidentiality, Integrity, Availability (CIA)**, and **Auditability**, which are core requirements of secure systems, especially in e-governance and critical decision-making platforms.

2. Project Objectives:

The objectives of the SecureVote are divided into **Security Objectives, Development Objectives**, and **Secure SDLC Phases Applied**, ensuring that security is treated as a core requirement rather than an afterthought.

2.1 Security Objectives

The primary security objective of this project is to design and implement a voting system that guarantees **trust, reliability, and protection of sensitive information**. Since voting systems handle critical user data and decision-making processes, strong security controls are mandatory.

1. Confidentiality

Confidentiality ensures that sensitive information such as user credentials and voting choices remains private and inaccessible to unauthorized entities.

Methods Used:

- Passwords are stored using strong one-way hashing (`password_hash()`).
- User sessions are managed securely using PHP sessions.
- Voting choices are never exposed on the frontend or through URLs.
- Backend files are isolated from direct user interaction.

Outcome:

Only authenticated and authorized users can access protected resources, and sensitive data remains confidential at all times.

2. Integrity

Integrity ensures that votes, election data, and system records cannot be altered, duplicated, or manipulated.

Methods Used:

- Database-level **foreign key constraints** enforce valid relationships.
- Backend logic prevents **double voting** by checking vote existence.
- Prepared statements are used to prevent SQL injection attacks.
- Election status controls ensure votes are only cast in open elections.

Outcome:

Each voter can cast only one vote per election, and all data remains accurate and tamper-resistant.

3. Availability

Availability ensures that the system remains accessible and functional for legitimate users during election periods.

Methods Used:

- Lightweight architecture with minimal dependencies.
- Efficient database queries.
- Role-based access to reduce unnecessary processing.

Outcome:

The system remains stable and usable under normal operational conditions.

4. Authentication and Authorization

Authentication verifies user identity, while authorization ensures users can only perform actions allowed by their role.

Methods Used:

- Secure login mechanism using hashed password verification.
- Role-Based Access Control (RBAC) implemented for Admin and Voter roles.
- Session validation on every protected page.

Outcome:

Unauthorized access is prevented, and users operate strictly within their permitted scope.

5. Auditability and Accountability

Auditability ensures that important system actions can be traced and reviewed when necessary.

Methods Used:

- Audit logs record critical actions such as election creation and vote casting.
- Each log entry includes user identity and action description.

Outcome:

The system supports accountability and post-event security analysis.

2.2 Development Objectives

The development objectives focus on building a secure, maintainable, and modular system while following best practices of secure software engineering.

1. Secure-by-Design Development

Security is integrated into every development stage instead of being added later.

Approach:

- Backend-first security enforcement.
- Minimal trust in frontend input.
- Defensive programming techniques.

2. Modular and Maintainable Architecture

The system is designed to be easy to understand, extend, and maintain.

Approach:

- Separation of frontend and backend logic.
- Centralized authentication and authorization modules.
- Reusable database connection and audit components.

3. Prevention of Common Web Vulnerabilities

The system proactively defends against widely known web application threats.

Addressed Threats:

- SQL Injection → Prepared statements
- Broken Authentication → Secure sessions and hashing
- Privilege Escalation → Role checks
- Data Tampering → Backend validation

4. Academic and Industry Relevance

The project is developed to reflect both academic learning outcomes and real-world security practices.

Outcome:

The system serves as a practical demonstration of secure software development principles.

2.3 Secure SDLC Phases Applied

The Secure Software Development Life Cycle (Secure SDLC) ensures that security is embedded throughout the development process.

Phase 1: Requirement Analysis

Security Focus:

- Identification of threats such as double voting, unauthorized access, and data leakage.
- Definition of security requirements including CIA triad and audit logging.

Result:

Clear security requirements were established before development began.

Phase 2: Secure System Design

Security Focus:

- Role-Based Access Control (RBAC).
- Layered system architecture.
- Secure database schema with constraints.

Result:

The system was architected to minimize attack surface and enforce security boundaries.

Phase 3: Secure Implementation

Security Focus:

- Use of prepared SQL statements.
- Password hashing and verification.
- Backend-only enforcement of business logic.

Result:

Security mechanisms were correctly implemented at the code level.

Phase 4: Testing and Validation

Security Focus:

- Testing unauthorized access attempts.
- Verifying prevention of double voting.
- Validating role-based restrictions.

Result:

Security controls were tested and verified for correctness.

Phase 5: Deployment and Review

Security Focus:

- Secure file structure.
- Restricted backend access.
- Manual security review.

Result:

The system was deployed in a controlled and secure environment.

3. System Architecture

The **SecureVote** follows a **layered architecture** that separates concerns between presentation, business logic, and data management. This approach enhances maintainability, security, and scalability, ensuring that sensitive operations and data are protected from unauthorized access.

3.1 Architecture Overview

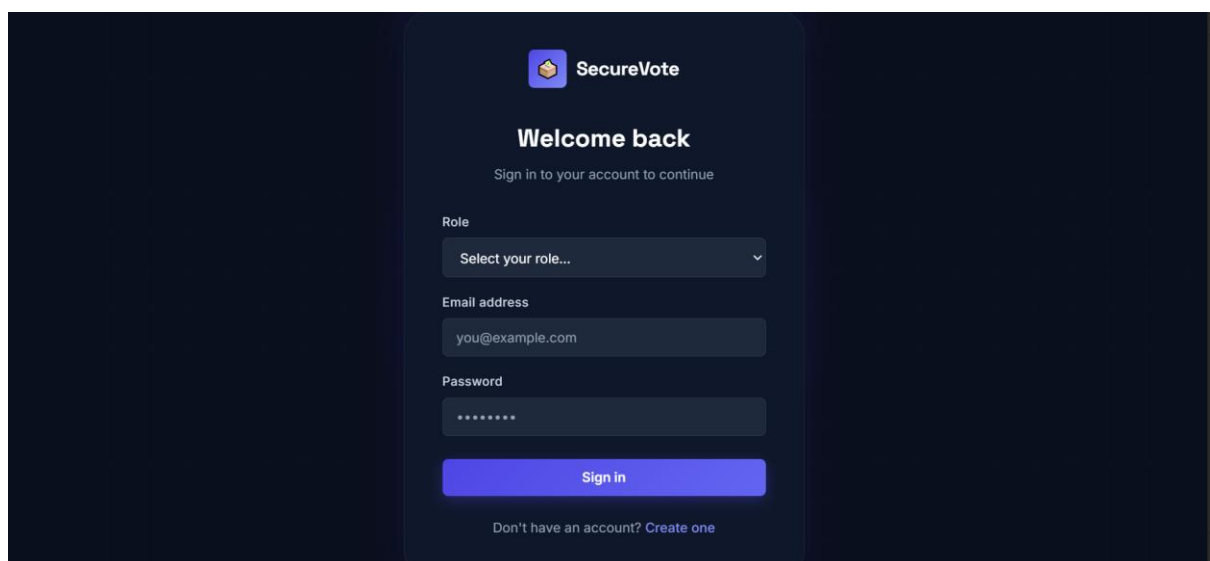
The system is organized into three main layers:

1. **Presentation Layer (Frontend)**
2. **Application Layer (Backend)**
3. **Data Layer (Database)**

Each layer has a clearly defined role and communicates securely with the adjacent layers.

3.2 Presentation Layer (Frontend)

Login:

A screenshot of the SecureVote login interface. The form is centered on a dark background. At the top, there is a logo with a ballot box icon and the text "SecureVote". Below the logo, the text "Welcome back" is displayed in a large, bold font, followed by the subtitle "Sign in to your account to continue" in a smaller font. The form contains three input fields: "Role" with a dropdown menu showing "Select your role...", "Email address" with the placeholder "you@example.com", and "Password" with masked characters "*****". A prominent blue "Sign in" button is located below the password field. At the bottom of the form, there is a link that says "Don't have an account? Create one".

 **SecureVote**

Welcome back

Sign in to your account to continue

Role

Select your role... ▼

Email address

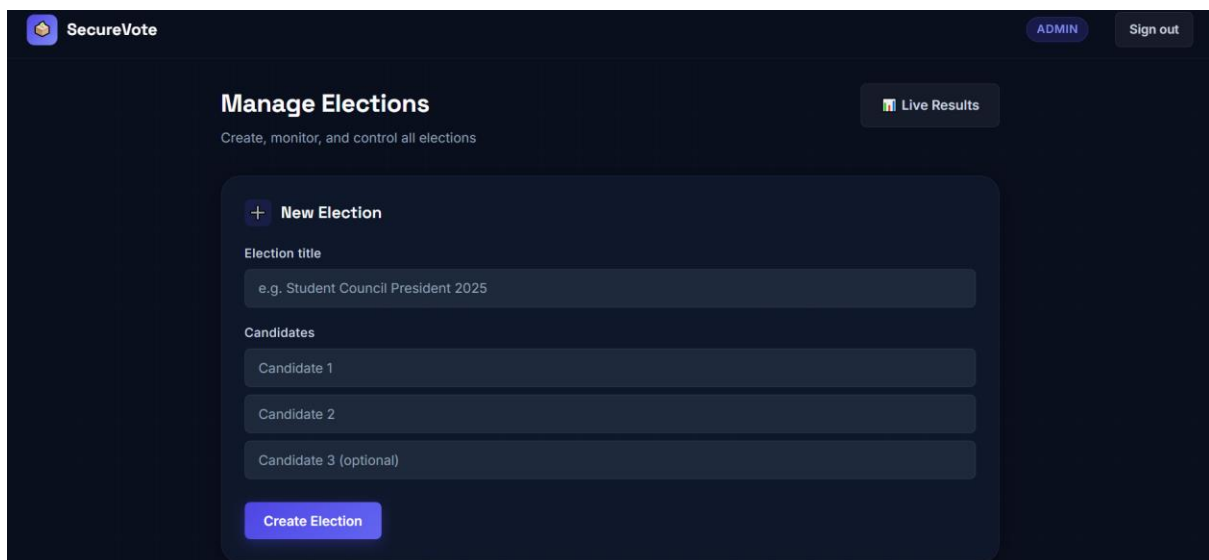
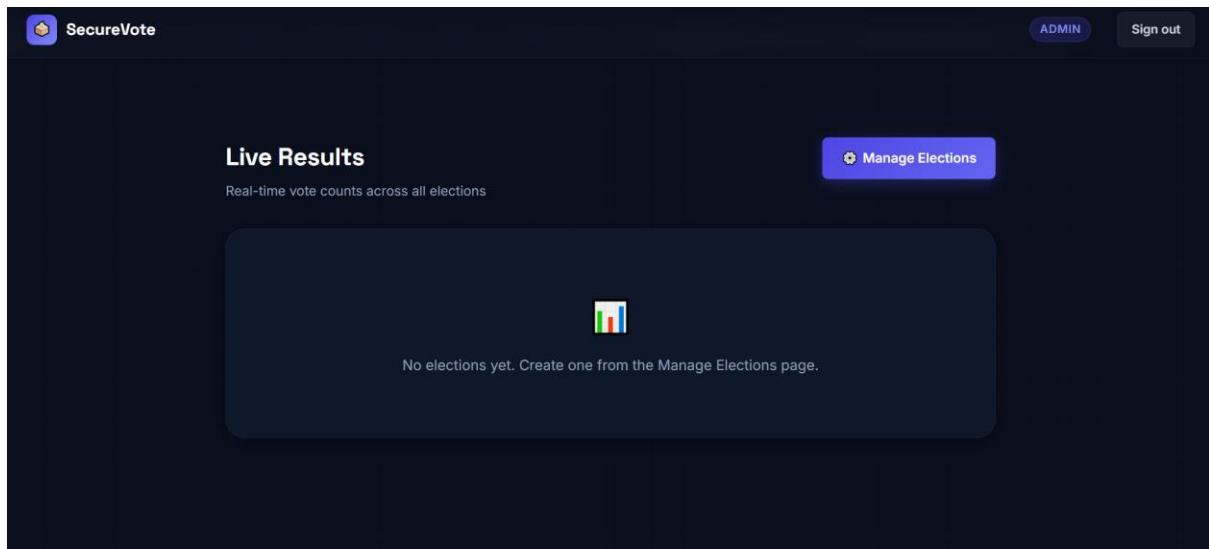
you@example.com

Password

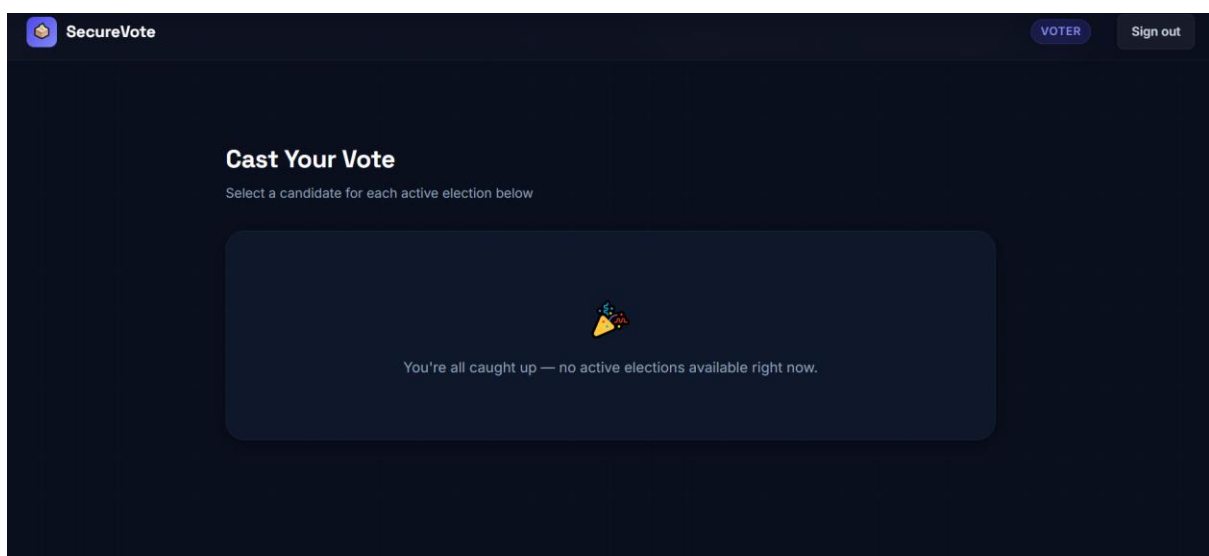
Sign in

Don't have an account? [Create one](#)

Admin:



Voter:



Responsibilities:

- Provides the user interface for Admins and Voters.
- Collects input via forms for login, signup, voting, and election management.
- Displays results, success/error messages, and election information.
- Ensures minimal sensitive processing occurs on the client side.

Technologies Used:

- HTML & CSS for UI
- Form validation for better UX

Security Measures:

- All sensitive logic occurs on the backend; frontend only handles display.
- Form inputs are validated before submission to reduce malformed requests.
- Direct access to backend files is restricted; users can only interact through approved frontend forms.

3.3 Application Layer (Backend)**Responsibilities:**

- Handles authentication and session management.
- Implements role-based access control (RBAC) for Admins and Voters.
- Processes votes securely, ensuring each voter votes only once per election.
- Manages CRUD operations for elections and candidates.
- Logs audit trails for critical operations like election creation and vote casting.

Technologies Used:

- PHP for server-side scripting
- Prepared statements for database queries
- Session-based authentication

Security Measures:

- Passwords are hashed using `password_hash()` and verified using `password_verify()`.
- Prepared statements prevent SQL injection attacks.
- Double-vote prevention is enforced by backend logic and database constraints.
- Audit logs record all important actions for accountability.
- Role checks prevent unauthorized access to restricted pages.

3.4 Data Layer (Database)**Responsibilities:**

- Stores all persistent data for users, elections, candidates, votes, and audit logs.
- Enforces data integrity using primary and foreign keys.

- Ensures referential integrity between elections, candidates, and votes.

Database Structure Highlights:

- **Users Table:** Stores user credentials and roles.
- **Elections Table:** Stores election details, including status (`open/closed`).
- **Candidates Table:** Stores candidates linked to elections.
- **Votes Table:** Stores votes, with a unique constraint on (`voter_id`, `election_id`) to prevent double voting.
- **AuditLogs Table:** Stores logs for all critical operations.

Security Measures:

- Foreign key constraints maintain referential integrity.
- Unique constraints prevent logical attacks like double voting.
- Backend-only access prevents direct modification by unauthorized users.

3.5 System Flow

1. **User Authentication:** Users (Voter/Admin) log in through the frontend. The backend verifies credentials and initiates a secure session.
2. **Role-Based Redirection:**
 - Admin → Admin Dashboard / Manage Elections
 - Voter → Voting Page (only active elections, candidates available)
3. **Election Management (Admin):** Admin can create elections and candidates. Actions are recorded in audit logs.
4. **Voting (Voter):** Voter can select a candidate for each election. Backend prevents duplicate votes and logs the action.
5. **Audit & Reporting:** All critical actions are logged for monitoring and accountability.

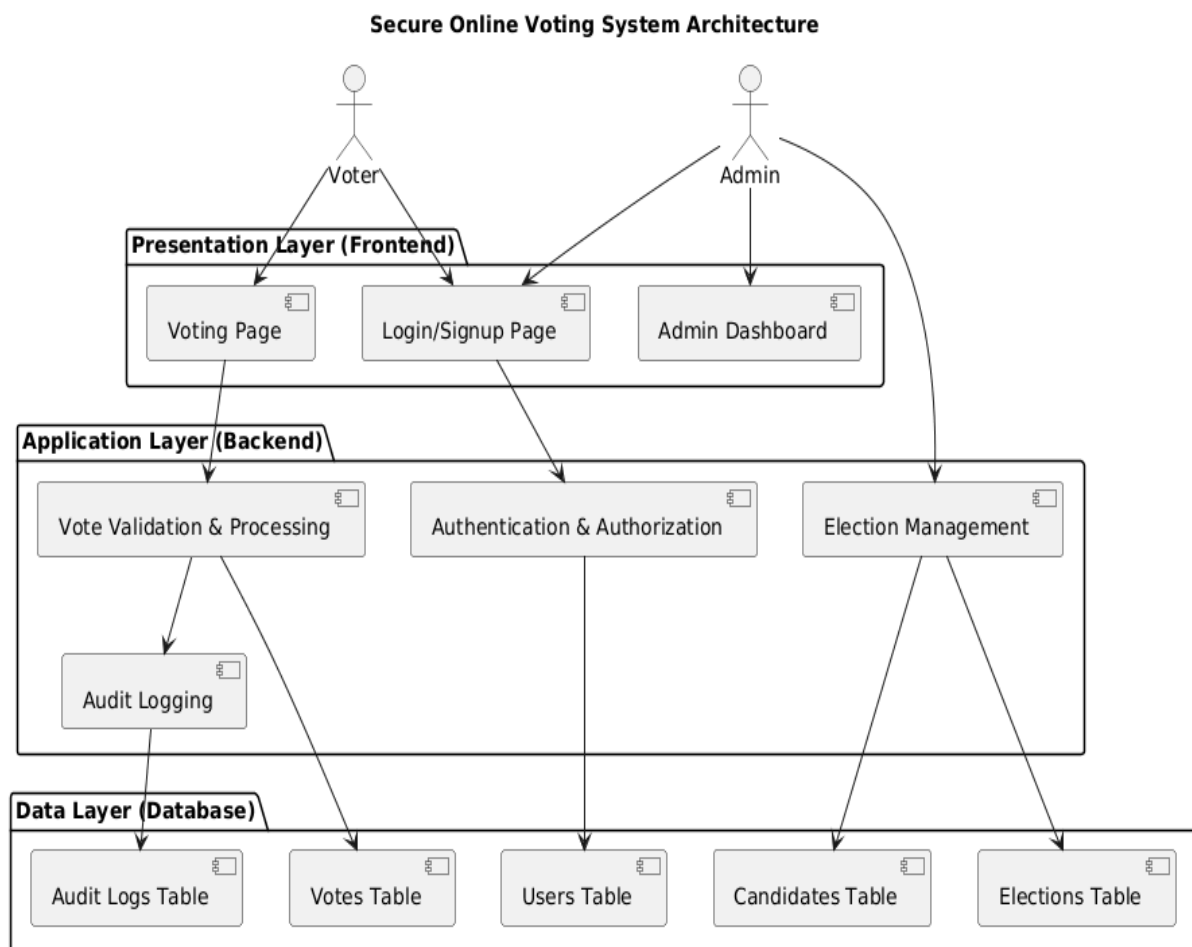
3.6 Security-Oriented Architecture Features

Layer	Security Feature	Implementation
Frontend	Input Validation	HTML5 required fields, JS validation
Backend	Authentication	Session-based login, password hashing
Backend	Authorization	Role-based access control (RBAC)
Backend	Integrity	Prepared statements, double-vote prevention
Database	Referential Integrity	Foreign keys, UNIQUE constraints
Backend	Audit & Non-repudiation	AuditLogs table, all critical actions recorded
Full System	Confidentiality	Sensitive logic and data reside only in backend

3.7 Advantages of This Architecture

- **Separation of Concerns:** UI, business logic, and data are separated, reducing vulnerabilities.
- **Scalable & Maintainable:** Easy to add new features like real-time results or election closing.
- **Security-Centric:** Backend-only sensitive operations, database constraints, and auditing ensure CIA and accountability.

Layout:



4. Design Diagrams

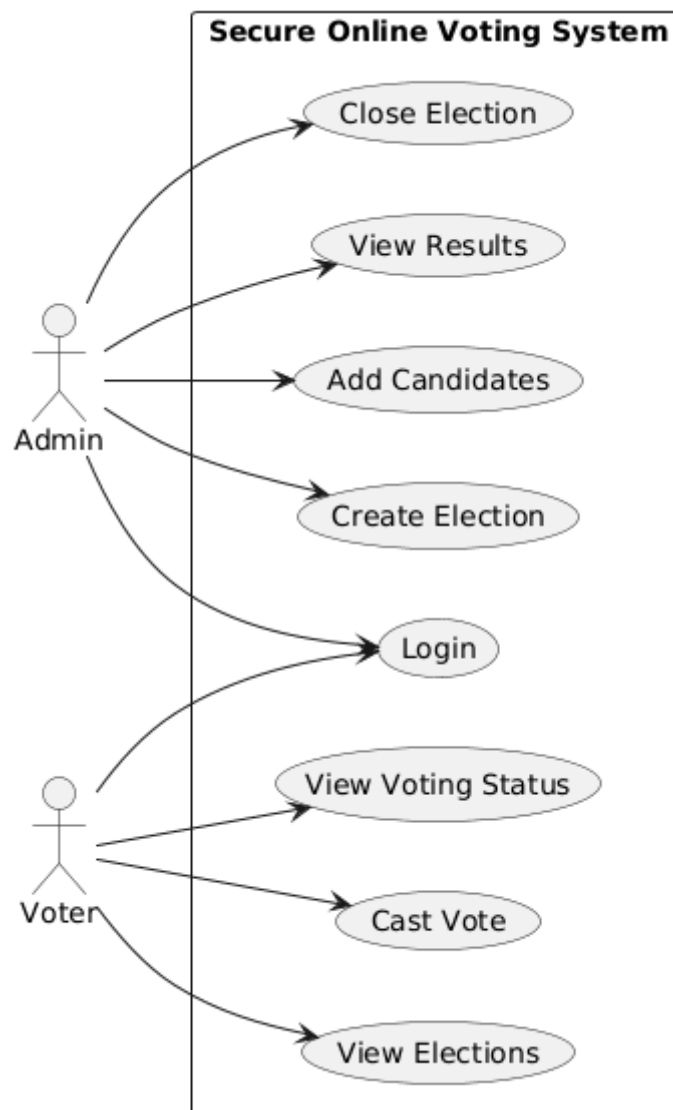
4.1 Use Case Diagram:

Actors:

- Admin
- Voter

Use Cases:

- Login / Logout
- Admin: Create Election, Add Candidates, View Results
- Voter: View Active Elections, Cast Vote



4.2 Entity Relationship Diagram (ERD)

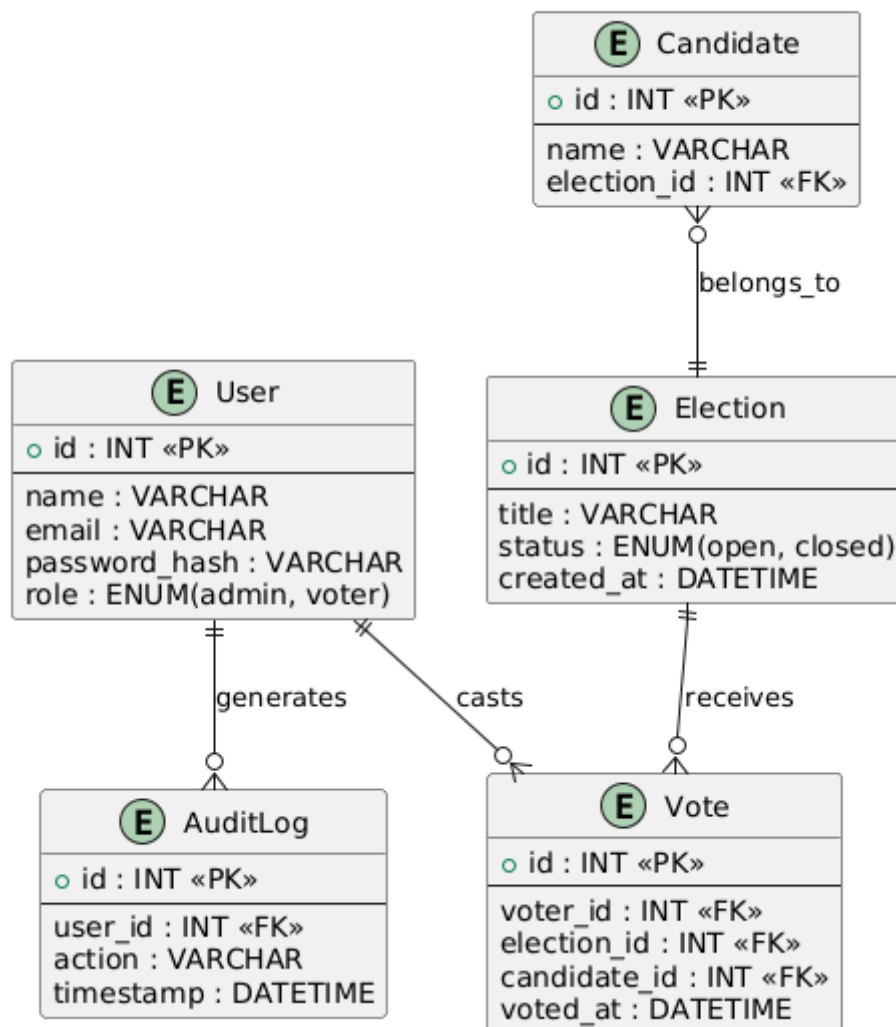
Entities and Relationships:

- User (id, name, email, password, role)
- Election (id, title, status)
- Candidate (id, name, election_id)
- Vote (id, voter_id, election_id, candidate_id)
- AuditLog (id, user_id, action, timestamp)

Relationships:

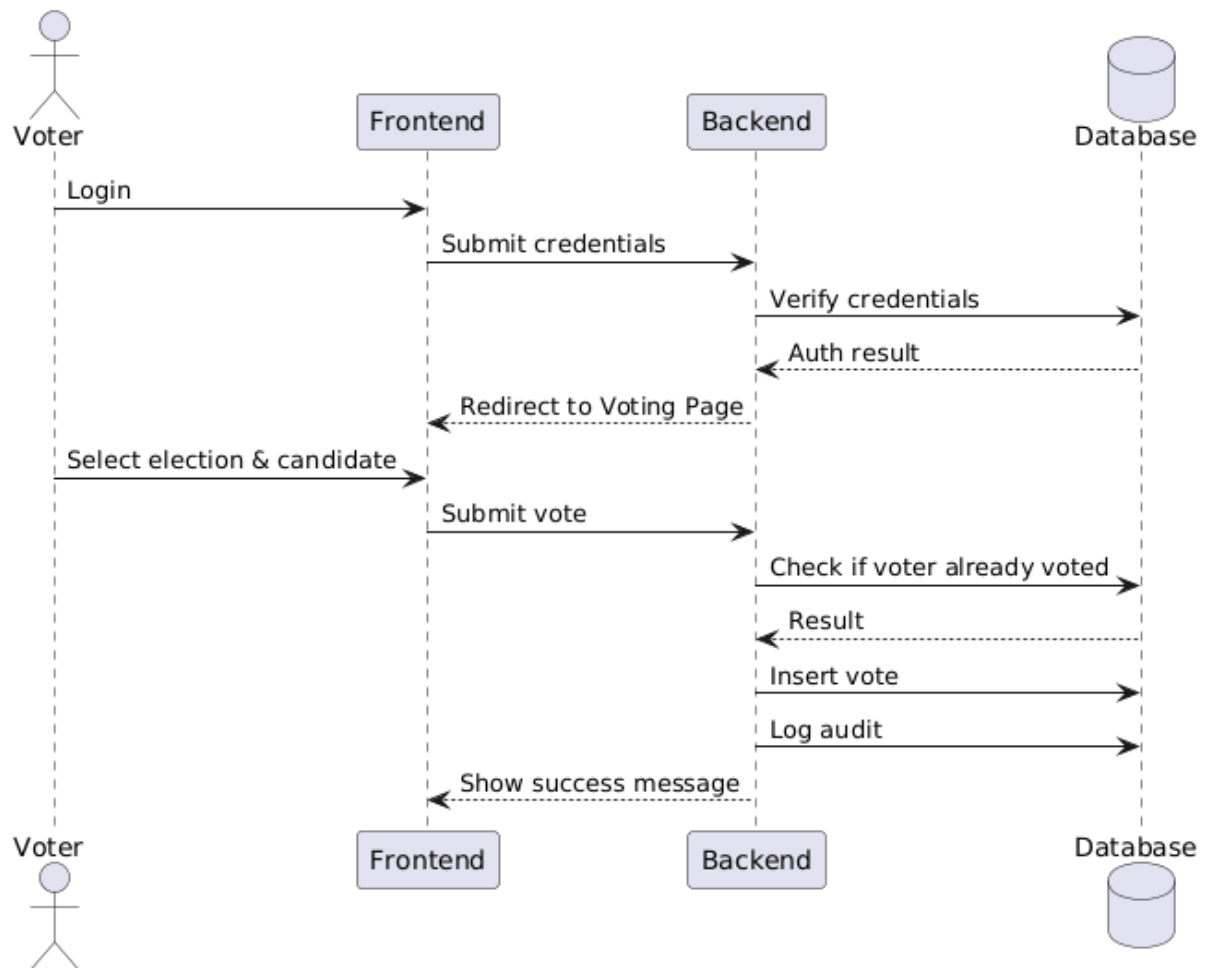
- One Election → Many Candidates
- One Voter → Many Votes (restricted to one per election)

Entity Relationship Diagram - Secure Online Voting System

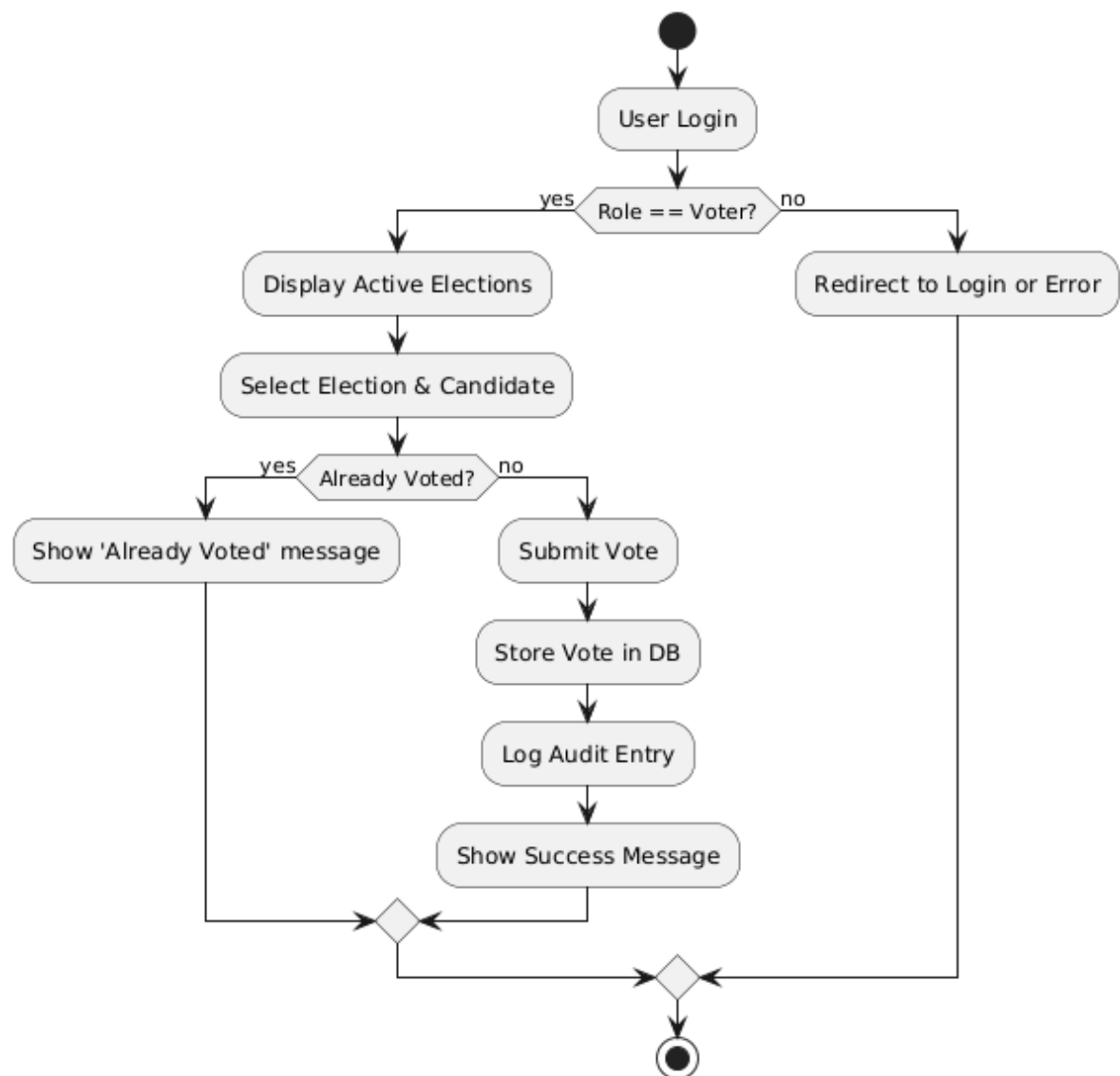


4.3 Data Flow Diagram (DFD - Level 1)

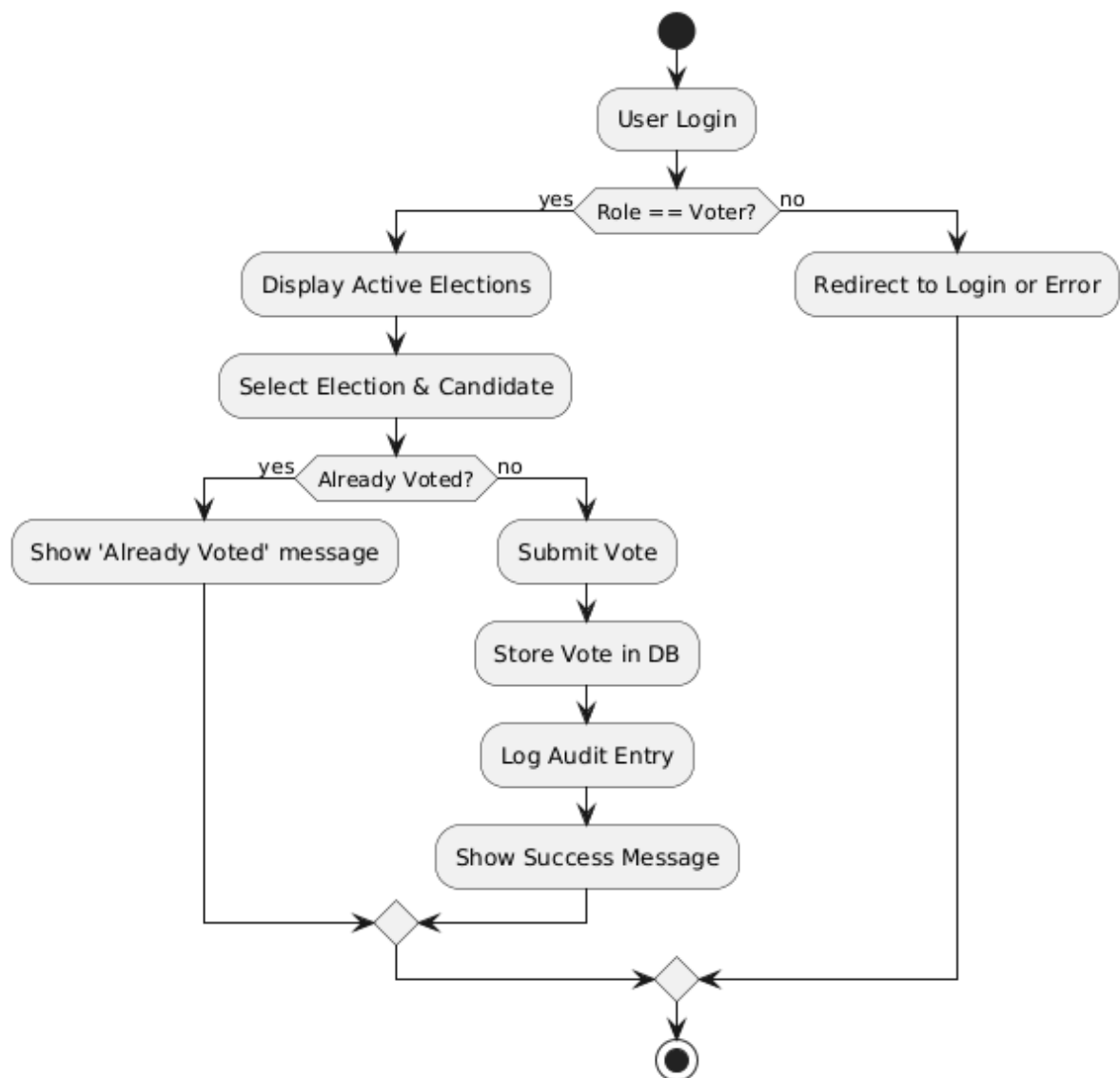
1. User submits credentials
2. Backend validates and authenticates
3. Role-based redirection
4. Voter submits vote
5. Backend validates & stores vote
6. Audit log created



4.4 Sequence Diagram



4.5 Activity Diagram



5. Working Plan (Phased Development Approach)

The SecureVote was developed using a **phased development approach**, following the principles of a **Secure Software Development Life Cycle (Secure SDLC)**. Each phase was planned to ensure that security and functionality were embedded from the beginning and validated at every stage.

Phase 1: Requirement Analysis

Objective: Identify system requirements and potential security threats.

Activities Performed:

- **Identified Security Threats:**
 - Possibility of **double voting** by voters.
 - Unauthorized access to admin or voter functionalities.
 - SQL injection or data tampering attempts.
 - Data confidentiality breaches (e.g., exposure of votes or passwords).
- **Defined Roles and Access Levels:**
 - **Admin:** Can create and manage elections, add candidates, view results.
 - **Voter:** Can view active elections and cast votes.
 - Roles were clearly defined to implement **Role-Based Access Control (RBAC)** in later phases.

Outcome:

A detailed list of functional and security requirements was created, forming the foundation for a secure design.

Phase 2: Secure Design

Objective: Architect the system to enforce security while supporting functionality.

Key Design Decisions:

- **Role-Based Access Control (RBAC):**
 - Each role has specific permissions enforced by the backend.
 - Admin pages are inaccessible to voters, and voting pages are restricted to voters only.
- **Backend-Only Business Logic:**
 - All critical operations such as vote validation, vote submission, and election management are handled only in the backend to prevent client-side manipulation.
- **Secure Directory Structure:**
 - Frontend files (HTML/CSS) separated from backend PHP scripts.
 - Sensitive files are placed in a secure backend directory to prevent direct access.

Outcome:

A clear, secure architecture that separates concerns and minimizes attack surfaces.

Phase 3: Implementation

Objective: Translate the secure design into a working system with robust security controls.

Activities Performed:

- **PHP Backend with Prepared Statements:**
 - All SQL queries use prepared statements to prevent SQL injection attacks.
- **MySQL Foreign Key Constraints:**
 - Relationships between tables (users, elections, candidates, votes) enforce **referential integrity**, preventing invalid data entry.
- **Password Hashing Using `password_hash()`:**
 - User passwords are stored in hashed format to ensure confidentiality.
 - Verified with `password_verify()` during login.

Outcome:

A functional system with built-in security mechanisms at the code and database levels.

Phase 4: Testing

Objective: Validate both functionality and security of the system.

Tests Conducted:

- **Unauthorized Access Attempts:**
 - Attempted to access admin pages with a voter session. Access was blocked successfully.



- _____
- **Double-Vote Prevention:**
 - Verified that a voter cannot submit more than one vote per election.
 - Both backend logic and database constraints were tested.
- **SQL Injection Resistance:**
 - Attempted common SQL injection patterns in login and form inputs.

- All inputs were safely handled using prepared statements.

Outcome:

System security validated, ensuring that major threats are mitigated.

Phase 5: Deployment & Review

Objective: Deploy the system and perform a final security validation.

Activities Performed:

- **Deployment on XAMPP (Apache + MySQL):**
 - The system was deployed locally for testing and demonstration purposes.
 - Frontend and backend directories were configured properly for security and access.
- **Manual Security Review and Validation:**
 - Reviewed session management, file paths, and access controls.
 - Verified that audit logging works correctly and sensitive data is protected.

6. Implementation Details

The implementation of the **SecureVote** focuses on embedding security measures at every stage of development while ensuring proper functionality for both Admin and Voter roles. This section explains how key security principles—**Authentication, Authorization, Confidentiality, Integrity, Availability, and Auditability**—were implemented in the system.

6.1 Authentication & Authorization

Objective: Ensure that only legitimate users can access the system and perform actions permitted by their role.

Implementation:

- **Password Storage:**
User passwords are never stored in plaintext. Instead, they are hashed using PHP's built-in `password_hash()` function:

```
$password = password_hash($_POST['password'], PASSWORD_BCRYPT);
```

- **Password Verification:**
During login, the system verifies user credentials using `password_verify()`:



```
• password_verify($password, $hash);
```

- **Session-Based Authentication:**

Upon successful login, a secure session is initiated to maintain the authenticated state across pages.



```
session_start();
```

- **Role Checks on Every Protected Page:**

Each page checks the user's role (Admin or Voter) to prevent unauthorized access. For example, a voter cannot access the admin dashboard.



```
if ($_SESSION['role'] !== 'admin') {  
    die("Access denied");  
}
```



```
if (!isset($_SESSION['user_id']) || $_SESSION['role'] !== 'voter')  
{  
    header("Location: ../../login.php");  
    exit;  
}
```

Outcome:

This approach ensures **only authorized users access the system**, and users operate strictly within their allowed permissions.

6.2 Confidentiality

Objective: Protect sensitive information from unauthorized access.

Implementation:

- **No Plaintext Passwords:**

All passwords are hashed, preventing exposure even if the database is compromised.

- **Sensitive Operations Handled Only in Backend:**
Critical actions like vote validation, election creation, and vote submission occur only on the server side.
- **Restricted Access to Backend Files:**
Backend directories are structured so that frontend users cannot directly access PHP scripts; access is mediated through proper requests and role checks.

Outcome:

Sensitive user and system data remain confidential, maintaining trust in the platform.

6.3 Integrity Controls

Objective: Ensure that data remains accurate, complete, and tamper-proof.

Implementation:

- **One-Vote-Per-Election Enforcement:**
 - **Backend Logic:** Before accepting a vote, the system checks whether the voter has already voted in that election.

```
$stmt = $conn->prepare(
    "SELECT id FROM votes WHERE voter_id = ? AND election_id = ?"
);
$stmt->bind_param("ii", $voter_id, $election_id);
$stmt->execute();

if ($stmt->get_result()->num_rows > 0) {
    header("Location: vote.php?msg=already_voted");
    exit;
}
```

- **Database Constraints:** Foreign key and unique constraints prevent invalid or duplicate entries.
- **SQL Injection Prevention:**
All database interactions use **prepared statements**, which separate data from SQL commands and prevent malicious input from compromising the database.

Outcome:

Votes, elections, and user data remain **reliable and untampered**, even in the presence of malicious users.

6.4 Availability Measures

Objective: Ensure that the system remains functional and accessible during normal operation.

Implementation:

- **Minimal Dependencies:**
The system uses PHP and MySQL with standard libraries, reducing the likelihood of external service failures.
- **Efficient Queries:**
Database queries are optimized to reduce latency and server load.
- **Simple Architecture:**
A three-layer architecture (Presentation, Application, Data) reduces complexity, minimizes potential bugs, and limits the attack surface.

Outcome:

Authorized users can access the system reliably without interruptions.

6.5 Audit & Logging

Objective: Maintain accountability and traceability of all critical actions.

Implementation:



```
logAction($user['id'], "User Logged In");
```

- **Admin Actions Logged:**
Actions such as creating, updating, or deleting elections are recorded in the **AuditLogs** table.
- **Vote Casting Events Recorded:**
Each vote submission is logged, including the voter ID, election ID, and timestamp.
- **Audit Trail for Accountability:**
Logs allow administrators to review actions, detect suspicious activity, and ensure compliance with election rules.

Outcome:

The system provides a clear and secure **audit trail**, supporting non-repudiation and accountability.

Conclusion

This SecureVote demonstrates how **secure software design principles** can be practically implemented in a real-world web application. By integrating security throughout the SDLC, enforcing CIA principles, and applying industry best practices, the project fulfills the objectives of the **Secure Software Design and Development** course and serves as a strong foundation for secure system engineering.